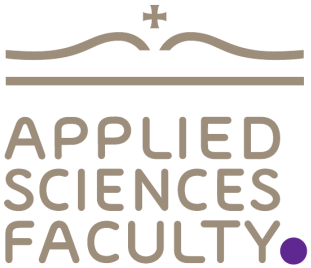
C++ — лекція 7. Функції — 2, поглиблені теми.





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Перевантаження функцій

- Різні функції[1], повинні мати різні імена.
- Однак, іноді трапляються цілі сімейства ідейно схожих функцій[2]. Для таких випадків є сенс використати одне і те ж ім'я. Зрозуміло, щоб компілятор міг їх відрізнити, у них повинні бути відмінні списки параметрів.
- Таке повторне використання імен функцій називається перевантаженням функцій.
- УВАГА! Компілятор не накладає якихось обмежень на те, що робитимуть перевантажені функції. Однак, категорично не рекомендується, щоб вони, в залежності від аргументів, робили щось ідейно різне — це заплутує структуру програми, робить її не-інтуїтивною.
- `https://quuxplusone.github.io/blog/2020/10/09/when-to-derive-and-overload/` – цікаві рекомендації[3].

[1] Які, відповідно, виконують різні задачі.

[2] Наприклад, обчислення абсолютного значення чисел різних типів.

[3] Коротко, ідея: перевантажувати чи використовувати успадкування, лише коли є якийсь код, який і не повинен знати, котру із перевантажених/перевизначених ф-цій він викликає. Дрібний виняток – оптимізовані для конкретних типів ф-ції. Продовження: `https://quuxplusone.github.io/blog/2020/10/11/overloading-considered-harmful/`.

Вибір перевантажених функцій

Зустрівши виклик перевантаженої функції, компілятор повинен вибрати, котрий із варіантів викликати. Відбувається відбір згідно наступних критеріїв співпадіння фактичних та формальних параметрів, у порядку застосування:

1. Точна відповідність типів, з точністю до тривіальних перетворень[4].
2. Відповідність може бути досягнута за допомогою "просування" інтегральних типів — приведення менш містких до більш містких[5].
3. Відповідність може бути досягнута за допомогою стандартних перетворень типів[6].
4. Відповідність може бути досягнута за допомогою перетворень типів, заданих користувачем.
5. Відповідність досягається з використанням трокрапки — засобу передачі функції довільної кількості параметрів.

- Додатково, між шаблонною та нешаблонною функцією, які однаково підходять, буде обрано не шаблонну – це не вважатиметься неоднозначністю. Поміж кількох шаблонних об'єктів, для виклику буде обрано найбільш конкретизований.

[4] Таких як приведення масиву до вказівника на нього, і `T` до `const T`.

[5] Наприклад, `short` до `int`.

[6] Наприклад, `int` в `double` чи `T*` в `void*`.

Неоднозначності

- Якщо виклик можна зробити кількома різними способами в рамках одного рівня критеріїв — виклик вважається неоднозначним, і виникає помилка компіляції.
- Наприклад, розглянемо функції піднесення до степені. Уявимо, що існують лише два варіанти перевантаження[7]:

```
double mypow (double base, double exponent);
   int mypow (int    base,    int exponent);
```

- Тоді виникає питання: виклик `mypow(2.0, 4)` це `mypow(double, double)` чи `mypow(int, int)`? Є неоднозначність, тому вибір перевантаженої функції не відбудеться.
- Звичайно, за допомогою явного приведення типів можна примусити вибрати конкретний варіант:

```
mypow (2.0, static_cast<double>(4) );
mypow (2.0, 4.0 ); // Як для літералу, більш лаконічно
```

Інший варіант — додати ще й таке перевантаження:

```
double mypow (double base, int exponent);
```

- Або ще простіше:

```
void fn(int );
void fn(double);
   ..............
short v = 5;
fn(v); // Неоднозначність.
```

[7] Я ігнорую [illegible]

Деякі технічні подробиці перевантаження

- **УВАГА!** Тип значення, що повертається функцією, не бере участі в перевантаженні.
- Тому не можна перевантажувати функції, що відрізняються лише ним.

Пошук кандидатів для перевантаження

- Пошук відбувається назовні по областях видимості, цей процес зупиняється, як тільки трапилося відповідне ім'я:

```
void fn();
void some_other_fn(){
    void fn(int x); // Локальне оголошення.
    fn(); // <=== Помилка компіляції!
}
```

Пошук до "void fn()" з охоплюючої області видимості не доходить, оскільки ім'я fn() трапилося швидше.

- Для просторів імен працює ADL[8] – пошук функцій у просторах імен, де оголошено її аргументи:

```
namespace N{
    struct T{};
    void fn(T){ cout << "N" << endl; }
}


.............
N::T t;
fn(t); // OK
```

- https://quuxplusone.github.io/blog/2021/10/29/

- Враховуючи правила вибору функції для перевантаження, можна сформулювати наступні правила[9]:
  - ◇ Повністю кваліфіковане ім'я `lib::func(arg);` означає – я точно знаю, яку функцію викликати, тип аргументу не може знати краще.
  - ◇ Конструкція виду: `using lib::func; func(arg);` стверджує – я знаю про функцію func, але тип аргументу може мати кращий варіант виконати цю дію.
  - ◇ Нарешті, `func(arg);` – поняття не маю, що це за func, але тип аргументу – знає.
- Приклад (сподіваюся, в стандартній бібліотеці – єдиний) – abs() в науковому коді, що працює із floating point, часто викличе abs(int), тому або std::abs(), або явно fabs().

[9] За мотивами `https://quuxplusone.github.io/blog/2022/04/14/another-reason-for-the-poison-pill/`. Сама стаття присвячена способам зламати цей підхід.

Аргументи по замовчуванню

Іноді частина аргументів функції використовується рідко, або ще тільки заплановано, що їх будуть використовувати майбутньому[10].

В таких ситуаціях можна використати аргументи по замовчуванню — їх значення вказується під час оголошення[11]:

```
void print_val(int val, int base = 10);
```

Того ж результату можна досягнути, використовуючи перевантаження:

```
void print_val(int val);
void print_val(int val, int base);
```

однак в таких ситуаціях воно веде до дублювання коду і маскує ідею, що це є одна і та ж функція, яка вміє модифікувати свою поведінку, але має один більш поширений спосіб використання.

[10] Але вказати потрібно, щоб коли вони стануть потрібними, не треба було змінювати код, що їх викликає.

[11] В різних оголошеннях вони повинні співпадати! В межах одиниці трансляції взагалі дозволяється вказувати значення по замовчуванню лише раз.

- Аргументи по замовчуванню можна вказувати тільки в кінці списку формальних аргументів:

```
void fn1(int a, int b, int c=3, int d=4); // OK

void fn2(int a, int b=2, int c, int d=4); // Error!
```

Best practices

- Багато Style guidelines радять уникати аргументі із значенням по замовчуванню на користь перевантаження функцій. Аргументація, див., наприклад: `https://google.github.io/styleguide/cppguide.html#Default_Arguments`.
- Аналіз їх недоліків: `https://quuxplusone.github.io/blog/2020/04/18/default-function-arguments-are-the-devil/`, щоправда, дещо упереджений.
- Cpp Core Guidelines навпаки, за можливості рекомендують їх використовувати замість перевантаження функцій: `https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-default-args`.
- Таких аргументів для віртуальних функцій потрібно уникати (або робити їх завжди однаковими – що ненадійно): `https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rh-virtual-default-arg`.
- Приклад отримання схожого синтаксису в C: `https://medium.com/@aliaksandr.kavalchuk/default-and-named-arguments-in-c-functions-9af8c4e34f9b`.

Змінна кількість аргументів — приклад

Стандартна бібліотека C містить функцію **printf**, яка може приймати не визначену наперед кількість аргументів:

```
int printf ( const char * format, ... );
```

Використовується ця функція так:

- Стрічка формату, format, містить текст для виводу, разом із керівними послідовностями, які визначають, як виводити змінні, що йдуть наступними аргументами.
- Після неї перераховано аргументи для виводу. **Увага**, компілятор не має ефективного методу перевірити, чи вказані в стрічці формату типи відповідають реально переданим!

Приклад:

```
int a = 5;
double b = 11.11;
char c = 'd';
char* s = "sbc";
printf ("int = %i, double = %g, char = %c, C-string = %s\n", a, b, c, s);
```

Детальніше про стрічку формату див.
http://www.cplusplus.com/reference/cstdio/printf/

Ілюстрація проблеми із безпекою типів у printf:

```
int a = 5;
double b = 11.11;
char c = 'd';
char* s = "sbc";

printf("int = %i, double = %g, char = %c, C-string = %s\n", a, b, c, s);

printf("int = %g, double = %i, char = %c, C-string = %s\n", a, b, c, s);
```

```
int = 5, double = 11.11, char = d, C-string = sbc
int = -8.67922e+209, double = 1076246609, char = d, C-string = sbc
```

- Другий випадок – невизначена поведінка, показано лише один з можливих результатів.
- `https://zhxnlai.github.io/printf/` – візуалізатор модифікаторів стрічки формату printf.

Реалізація функцій із змінною кількістю аргументів

- Така функція повинна містити принаймні один заданий наперед (фіксований) аргумент.
- Після списку фіксованих аргументів, замість чергового аргументу, знаходиться три крапки:

```
int varargsfn(int fixed1, int fixed2, ...);
```

- В файлі <cstdarg> знаходиться набір відповідних макросів.
- `va_list` — тип службової структури для збереження інформації про аргументи.
- `va_start()` — ініціалізує цю структуру.
- `va_arg()` — повертає черговий переданий параметр, із вказаним типом. Не вміє ні перевірити тип, ні визначити, чи аргумент є останнім.
- `va_end()` — завершує обробку списку аргументів. Слід викликати в тій же функції, де було викликано `va_start()`!

- GCC надає нестандартний атрибут sentinel, який дозволяє компілятору перевіряти, чи останній аргумент є NULL:

  void __attribute__((sentinel)) warning_if_missed_NULL(**int** arg, ...);

- sentinel(N) дозволяє вказати, що NULL має бути елемент номер N з кінця – sentinel це синонім до sentinel(0).
- Детальніше див. `https://gcc.gnu.org/onlinedocs/gcc/Common-Function-Attributes.html`.

Приклад:

```
//! Друкуємо передані числа, поки не зустрінемо нуль.
void prn_seq_till_0(const char *intro, ...)
{
    int c;
    va_list arglist;
    va_start(arglist, intro);
    cout << intro << endl;
    do
    {
        c = va_arg(arglist, int);
        cout << c << " ";
    }while(c!=0);
    cout << endl;
    va_end(arglist);
}

  .....

prn_seq_till_0("Numbers: ", 1,2,3,4,0);
```

Вказівники на функції

- Як і для масивів, ім'я функції насправді задає вказівник на функцію[12].

```
int add(int a, int b) {    return (a+b);    }
int sub(int a, int b) {    return (a−b);    }
int operation(int a, int b, int (*fn_ptr)(int,int)){
    int res = (*fn_ptr)(a,b);
    return res;
}    .............
    int x,y, z;
    int (*opcode)(int,int) = sub; // Можна й ... = &sub;

    z = opcode(3, 5);              // Можна і int z = (*opcode)(3, 5);
    x = operation (7, 5, add);
    y = operation (20, x, opcode);
```

- Типи аргументів функції та тип, що вона повертає, вказані під час оголошення вказівника повинні точно співпадати із типами функції, адреса якої йому надається.
- Через незручність такого оголошення вказівників на функцію, краще скористатися:

```
typedef int(*mathf_ptr_t)(int, int);    // Pre C++11
using mathf_ptr_t = int(*)(int, int);   // C++11
mathf_ptr_t minus = sub;
```

[12] Тому, зокрема, розіменовувати вказіваник на функцію чи явно брати її адресу за допомогою оператора & немає потреби.

Приклад:

Функція стандартної бібліотеки C, qsort:

```
void qsort (void* base, size_t num, size_t size,
            int (*compar)(const void*,const void*));
```

є дуже низькорівневою, зокрема — нічого не знає, про аргументи, які вона сортуватиме, тому їй слід передавати вказівник на область пам'яті із масивом даних для сортування, кількість елементів, розмір одного елемента цього масиву, та **функцію**, що отримавши вказівник на два елементи для порівняння, поверне від'ємне значення, якщо перший аргумент повинен бути перед другим, додатне – якщо навпаки, і 0, якщо елементи еквівалентні.

```
int compare (const void * a, const void * b)
{
    return ( *(int*)a - *(int*)b );
}

.....
    int arr []={5,4,3,2,1};
    const int arr_size=sizeof(arr)/sizeof(arr [0]);
    cout << "Before sort: ";
    for(int i = 0; i<arr_size; ++i )
    {
        cout << arr[i] << ",";
    }
    cout << endl;
    qsort (arr, arr_size, sizeof(int), compare);
    cout << "After sort: ";
    for(int i = 0; i<arr_size; ++i )
    {
        cout << arr[i] << ",";
    }
    cout << endl << endl;
```

inline-функції

Функцію можна визначати із специфікатором `inline`:

```
inline int multipy_by_2(int x){
    return 2*x;
}
```

- Він вказує компілятору, по можливості, вставляти код весь функції в місці її виклику, на противагу власне здійсненню виклику.
- З точки зору семантики, поведінка програми при цьому не змінюється!
- Зокрема, inline-функція має свій адрес — якщо в ньому виникне потреба, крім вбудованих (inline) версій, компілятор також згенерує звичайну версію, для виклику.
- Компілятор, з тих чи інших міркувань[13], може знехтувати цим специфікатором.
- Щоб вбудовування (inlining) функції відбувався успішно, компілятор повинен бачити не тільки її оголошення, але і визначення. Тому їх, на відміну від "звичайних", не-inline, функцій, слід оголошувати у відповідних файлах заголовків, (".h-файлах").

[13] Наприклад, продуктивності.

Керування інлайнингом

- Компілятори виконують інлайнинг автоматично і доволі добре, незалежно від цього модифікатора.
- ◇ При тому, компілятор може його ігнорувати.
- ◇ Щоб компілятор міг виконати інлайнинг, він має бачити і функцію і місце виклику, тому, за наявності кількох одиниць трансляції[14], варто використовувати LTO – Link-Time Optimization.
- Тому, inline не бажано використовувати – зараз воно швидше "функція, яку можна вписати в файли заголовків".
- З іншого боку, компілятори надають нестандартні засоби контролю за інлайнингом. Наприклад:
- ◇ `__attribute__((always_inline))` – завжди інлайнити:

```
__attribute__((always_inline)) inline void fn(int arg) { ..... }
```

- ◇ `__attribute__((flatten))` – інлайнити всі функції, які викликає ця.

[14] Кажучи менш строго – є кілька .cpp файлів.

Керування інлайнингом

- Також, допомагають оптимізації такі атрибути:
  - `__attribute__((pure))` – функція чиста, не має побічних ефектів.
  - `__attribute__((const))` – функція чиста, і її результат залежить лише від аргументів функції.
  - `__attribute__((cold))` – функція використовується рідко.
- Детальніше див. “gcc: Common Function Attributes”.

Посилання

- Про роботу лінкера, та, чому він не може так просто діагнозувати, що тіла інлайн функції різні у різних одиницях трансляції: "A common proposed solution to certain categories of IFNDR: Getting the linker to verify identical functions".
- "Inlining - the ultimate optimisation", "Partial inlining" – приклади.

int main()

- main — теж **функція**.
- Функція з прототипом, одним із:

```
int main() {...}
int main(void) {...}
int main(int argc, char *argv[]) {...}
int main(int argc, char **argv) {...}
/* Нестандартне розширення: */
int main (int argc, char *argv[], char *envp[]) {...}
```

  є точкою входу в програму. При запуску код починає виконуватися саме з неї.
- Деякий код може виконуватися до main, готуючи середовище для неї.
- Значення, що повертається з main, служить кодом завершення. Воно може використовуватися програмою, яка викликала дану, для прийняття рішення про подальші дії. Традиційно вважається, що код повернення 0 позначає успішне завершення програми, будь-який інший — помилку.

Приклад роботи із аргументами командного рядка

```
#include <iostream>

using namespace std;

int main(int argc, char *argv[] )
{
    int i;
    cout << "Arguments: " << argc << endl;
    for(i=0;i<argc;++i)
    {
        cout << "Argument " << i << ": " << argv[i] << endl;
    }

    return 0;
}
```

Приклад роботи із змінними середовища

```
#include <iostream>

using namespace std;

int main(int argc, char **argv, char** envp){
    char** env;
    cout << "Environment variables:" << endl;
    for (env = envp; *env != 0; env++) {
        char* thisEnv = *env;
        cout << thisEnv << endl;
    }
    return 0;
}
```

- Стандартний спосіб – оголосити глобальну змінну environ.

  ```
  char **environ;
  ```

- Її роль та ж, що й нестандартного третього аргументу main().
- Детальніше див. `https://man7.org/linux/man-pages/man7/environ.7.html`.
- Підтримується і під MSVC.

Посилання

- `https://mariusbancila.ro/blog/2022/01/01/the-evolution-of-functions-in-modern-cpp/` – перераховано види функцій в C++.
- "Make your programs run faster: avoid function calls" – в значній мірі про інлайнинг.

